QUANTUM REJECTION SAMPLING FOR NEURO-SYMBOLIC AI

RESEARCH NOTES IN THE ENEXA AND QROM PROJECTS

March 13, 2026

ABSTRACT

We investigate a synergy between explainability in Neuro-Symbolic AI, reflected in tensor network structure, and sparse Quantum Circuits preparing the models. In particular we study a quantum rejection sampling scheme to prepare samples from Computation-Activation Networks. For experiments and prototyping we introduce the python library qcreason.

Contents

1 Introduction **2**

1.1 Related works . . . 2

1.2 Contributions . . . 2

2 Notation **2**

2.1 Tensor networks . . . 2

2.2 Computation-Activation Networks . . . 3

2.3 Q-samples . . . 3

3 Computation circuits **3**

3.1 Relation with basis encodings . . . 4

3.2 Decomposition sparsity . . . 5

3.3 Polynomial sparsity . . . 5

4 Activation circuits **7**

4.1 Uniformly controlled rotations . . . 7

4.2 Ancilla augmentation . . . 8

4.3 Q-samples of CompActNets . . . 8

5 Quantum Rejection Sampling from Computation-Activation Networks **10**

5.1 Amplitude Amplification . . . 10

5.2 Sampling from Computation-Activation Networks . . . 11

6 Conclusion and Outlook **11**

A Implementation 13
A.1 Quantum Circuits 13
A.2 Generic Contraction 13

1 Introduction

Neuro-Symbolic AI can be approached in the tensor network formalism tnreason Goessmann et al. (2026), unifying logical, neural and probabilistic paradigms of artificial intelligence. Quantum Circuits are also, by the axioms of quantum mechanics, tensor networks. This motivates this work, which investigates quantum algorithms for inference leading to efficiency increases compared with classical alternatives. We study an rejection sampling inference scheme of the broad model class. Quantum rejection sampling then brings an square root quantum advantage. To be more precise, we consider Computation-Activation Networks, a generic model class in tnreason fo Neuro-Symbolic AI.

1.1 Related works

Quantum rejection sampling has been developed in these works:

- Grover (1996) Grover algorithm (search in unstructured database)
- Brassard and Hoyer (1997); Brassard et al. (2002) generalization to amplitude amplification
- Ozols et al. (2013) introduced quantum rejection sampling (using amplitude amplification)
- Low et al. (2014) used quantum rejection sampling for Bayesian network sampling (which is NP-hard when conditioned on evidence, see e.g. Koller and Friedman (2009))
- No exponential speedups expected by Quantum Computing in AI Bennett et al. (1997)

Several applications of Quantum Inference on symbolic AI have been studied. The novelty of our approach is a concise framework extending these approaches and connecting with generic neural decompositions of functions (see computation circuits).

Main approach for sampling: Quantum Inference scheme on Bayesian networks Low et al. (2014)

- Extend to more general tensor networks than Bayesian networks: Computation-Activation Networks

Further literature:

- Schuld and Petruccione (2021): Sect 7.3.2 Reviewing the paper Low et al. (2014) as an application of fault-tolerant quantum computing
- Wittek and Gogolin (2017): Review of Markov Logic Network sampling (which are a special case of Computation-Activation Networks)

1.2 Contributions

We apply an adjusted quantum rejection sampling approach of Low et al. (2014) for Computation-Activation Networks Goessmann et al. (2026). The main adjustments are:

- **Sparse representation of uniformly controlled unitaries:** We apply similar sparsity mechanisms as for classical Computation-Activation Networks to decompose the uniformly controlled unitaries.
- **Ancilla augmentation:** To sample also from generic tensor networks, we introduce ancilla variables such that generic tensor networks are conditioned Bayesian networks.

2 Notation

2.1 Tensor networks

We here use the tensor network notation Goessmann et al. (2026). The modes of each tensors are assigned by variables, and denoted in brackets as

$$\tau\left[X_{[d]}\right] .$$

Quantum gates are along this line unitary tensors with incoming and outgoing variables. The Pauli-X for example is denoted by

$$\sigma_1 \left[A^{\text{in}}, A^{\text{out}}\right] := \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \left[A^{\text{in}}, A^{\text{out}}\right] .$$

Contractions are denoted by angle brackets with the variables left open specified in further brackets. Circuits are contractions of the unitaries, leaving only the last output variables open. Computational basis measurements of circuits are coordinatewise squared absolut of the contracted circuit.

2.2 Computation-Activation Networks

Computation-Activation Networks (CompActNets) are tensor networks representing logical, probabilistic and neural models Goessmann et al. (2026). We here introduce them for Boolean statistics.

To any Boolean function $f : [2]^d \to [2]$ we define its basis encoding

$$\beta^f \left[Y, X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} \epsilon_{f(x_{[d]})} \left[Y\right] \otimes \epsilon_{x_{[d]}} \left[X_{[d]}\right] ,$$

where we denote the basis vectors as $\epsilon_0 \left[X\right] = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \left[X\right]$ and $\epsilon_1 \left[X\right] = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \left[X\right]$.

Definition 1 (Computation-Activation Network (CompActNets))**.** *Let there be a function $t : [2]^d \to [2]^p$ with basis encoding $\beta^t \left[Y_{[p]}, X_{[d]}\right]$. Let there further be a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with nodes $\mathcal{V}$ containing $[p]$. We define the by t computable and by $\mathcal{G}$ activated family of distributions by*

$$\Lambda^{t,\mathcal{G}} = \left\{ \left\langle \beta^t \left[Y_{[p]}, X_{[d]}\right], \langle \xi \rangle_{\left[Y_{[p]}\right]} \right\rangle_{\left[X_{[d]} | \varnothing\right]} : \xi \left[Y_{\mathcal{V}}\right] \in \mathcal{T}^{\mathcal{G}} \right\} .$$

We refer to any member $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{t,\mathcal{G}}$ as a Computation-Activation Network *(or shorter as a* CompActNet*). We call $\beta^t \left[Y_{[p]}, X_{[d]}\right]$ (and any decomposition of it) the* computation network *and $\xi \left[Y_{\mathcal{V}}\right]$ the* activation network.

2.3 Q-samples

In general, we define q-samples to be quantum states, which measured in the computational basis reproduce a given probability distribution.

Definition 2 (Q-sample **?**Low et al. (2014))**.** *Given a probability distribution $\mathbb{P} : \bigtimes_{k \in [d]} [2] \to \mathbb{R}$ (i.e. $\langle \mathbb{P} \rangle_{[\varnothing]} = 1$ and $0 \prec \mathbb{P}$) its q-sample is*

$$\psi^{\mathbb{P}} \left[X_{[d]}\right] = \sum_{x_{[d]} \in \bigtimes_{k \in [d]} [m_k]} \sqrt{\mathbb{P}\left[X_{[d]} = x_{[d]}\right]} \cdot \epsilon_{x_{[d]}} \left[X_{[d]}\right] .$$

Q-samples are more restrictive than arbitrary states having a distribution as a measurement distribution, since they demand real and positive amplitudes.

Example 1 (Q-sample of the uniform distribution)**.** *The q-sample of the uniform distribution can be prepared by Hadamard gates acting on the ground state. This is an example of a Walsh-Hadamard transform, which can be performed by unary Hadamard gates.*

Generalizing Example 1, Q-samples to independent distributions of boolean variables can be prepared by unary rotations along the y-axis.

3 Computation circuits

With our aim to sample from CompActNets, let us first investigate mechanisms to represent the computation network. More precisely, we construct sparse circuoits preparing q-samples to the normalized computation network

$$\mathbb{P}\left[Y_{[p]}, X_{[d]}\right] = \beta^t \left[Y_{[p]}, X_{[d]}\right] .$$

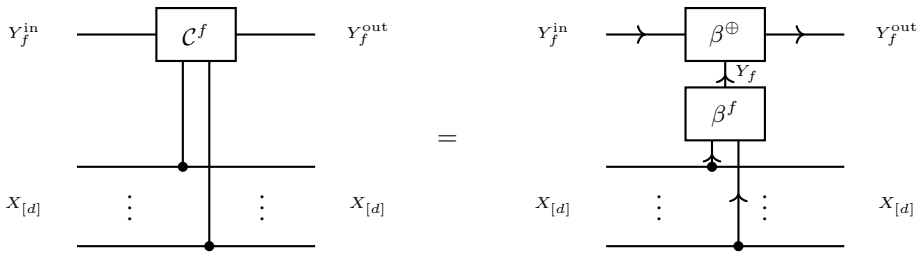

Figure 1: Relation of computation circuit and basis encodings. The computation circuit of a function is the contraction of its basis encoding with the basis encoding of the mod2 sum $\oplus$.

Definition 3 (Computation circuit)**.** *Given a boolean function $f : \bigtimes_{k\in[d]}[2] \to [2]$, its oracle or computation circuit is the unitary tensor*

$$\begin{aligned}\mathcal{C}^f\left[Y_f^{\text{in}}, Y_f^{\text{out}}, X_{[d]}\right] \quad = & \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]\,:\,f\left[x_{[d]}\right]=1} \sigma_1\left[Y_f^{\text{in}}, Y_f^{\text{out}}\right]\otimes\epsilon_{x_{[d]}}\left[X_{[d]}\right] \\ & + \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]\,:\,f\left[x_{[d]}\right]=0} \delta\left[Y_f^{\text{in}}, Y_f^{\text{out}}\right]\otimes\epsilon_{x_{[d]}}\left[X_{[d]}\right]\,.\end{aligned}$$

Example 2 (CNOT gate)**.** *The CNOT gate is the controlled Pauli-X, that is*

$$\text{CNOT}\left[Y^{\text{in}}, Y^{\text{out}}, X\right] = \sigma_1\left[Y^{\text{in}}, Y^{\text{out}}\right]\otimes\epsilon_1\left[X\right] + \delta\left[Y^{\text{in}}, Y^{\text{out}}\right]\otimes\epsilon_0\left[X\right]\,.$$

For the identity function

$$\text{Id}(x) = x\,.$$

we have

$$\mathcal{C}^{\text{Id}}\left[Y^{\text{in}}, Y^{\text{out}}, X\right] = \text{CNOT}\left[Y^{\text{in}}, Y^{\text{out}}, X\right]\,.$$

The CNOT *is therefore the computation circuit to the identity function.*

Functions with multiple output variables, i.e. $f : \bigtimes_{k\in[d]}[2] \to \bigtimes_{\ell\in[p]}[2]$, can be encoded image coordinate wise as a concatenation of the respective circuits.

3.1 Relation with basis encodings

Basis encodings are schemes to encode functions into directed boolean tensors, which enable reasoning about functions by tensor network contractions (see Goessmann et al. (2026)). To connect with this formalism, let us now show the relation to the computation circuits.

Lemma 1. *For any boolean function we have (see Figure 1)*

$$\mathcal{C}^f\left[Y_f^{\text{in}}, Y_f^{\text{out}}, X_{[d]}\right] = \left\langle\beta^f\left[Y_f, X_{[d]}\right], \beta^{\oplus}\left[Y_f^{\text{out}}, Y_f^{\text{in}}, Y_f\right]\right\rangle_{\left[Y_f^{\text{in}}, Y_f^{\text{out}}, X_{[d]}\right]}\,.$$

We here suggest a quantum pendant to basis encodings (see Chapter 14 in Goessmann (2025)), which has the decomposition by contraction property.

The computation circuit applied on the initial state

$$\sqrt{\frac{1}{2^d}}\mathbb{I}\left[X_{[d]}^{\text{out}}\right]\otimes\epsilon_0\left[Y^{\text{in}}\right]$$

prepares a q-sample to the normalized basis encoding.

In Goessmann et al. (2026) (and more detailed in Goessmann (2025)) sparse decompositions of basis encodings have been studied. Based on the equivalence of computation circuits and contracted basis encodings, stated by Lem. 1 we transfer these decomposition schemes in the following.

Example 3 (Multiple-controlled NOT)**.** *Let us consider the d-ary controlled* NOT *gate*

$$\mathrm{MCNOT}^d\left[Y^{\mathrm{in}}, Y^{\mathrm{out}}, X_{[d]}\right] = \sigma_1\left[Y^{\mathrm{in}}, Y^{\mathrm{out}}\right] \otimes \epsilon_{1_{[d]}}\left[X_{[d]}\right] + \delta\left[Y^{\mathrm{in}}, Y^{\mathrm{out}}\right] \otimes \left(\mathbb{I}\left[X_{[d]}\right] - \epsilon_{1_{[d]}}\left[X_{[d]}\right]\right) .$$

This is the computation circuit of the d-ary logical conjunction

$$\wedge^d : [2]^d \to [2] \quad , \quad \wedge^d(x_{[d]}) = \begin{cases} 1 & \textit{if} \quad x_{[d]} = 1_{[d]} \\ 0 & \textit{else} \end{cases} .$$

Special instances are the CNOT *for $d = 1$ (see Example 2) and the* Toffoli *gate for $d = 2$.*

3.2 Decomposition sparsity

The decomposition by contraction property of basis encodings is now a composition of circuits property, as stated in the next lemma. Instead of hidden variables as in the tnreason approach, we here exploit further working qubits.

Lemma 2. *We have for functions $q : \bigtimes_{k\in[d]}[2] \to \bigtimes_{\ell\in[p]}[2]$, $g : \bigtimes_{\ell\in[p]}[2] \to \bigtimes_{s\in[r]}[2]$ (see Figure **??**)*

$$\begin{aligned} &\mathcal{C}^{g\circ q}\left[Y^{\mathrm{in}}_{g\circ q}, Y^{\mathrm{out}}_{g\circ q}, X_{[d]}\right] \otimes \epsilon_0\left[Y^{\mathrm{out}}_q\right] \\ &\quad = \left\langle \epsilon_0\left[Y^{\mathrm{in}}_q\right], \mathcal{C}^q\left[Y^{\mathrm{in}}_q, Y^{\mathrm{mid}}_q, X_{[d]}\right], \mathcal{C}^g\left[Y_{\mathrm{in},g\circ q}, Y^{\mathrm{out}}_{g\circ q}, Y^{\mathrm{mid}}_q\right], \mathcal{C}^q\left[Y^{\mathrm{mid}}_q, Y^{\mathrm{out}}_q, X_{[d]}\right] \right\rangle_{\left[Y^{\mathrm{in}}_{g\circ q}, Y^{\mathrm{out}}_{g\circ q}, X_{[d]}, Y^{\mathrm{out}}_q\right]} . \end{aligned}$$

Here each copy of $Y_{g\circ q}$ denotes a tuple of r boolean variables and Y_q of p boolean variables.

Proof. We show the claim based on basis calculus and sketch the steps in Figure 2. In the first equation we use Lem. 1 and represent each computation circuit by a contraction of two basis encodings. In the second equation we use the identity

$$\left\langle \beta^{\oplus}\left[Y^{\mathrm{out}}, Y^{\mathrm{in}}, Y_q\right], \epsilon_0\left[Y^{\mathrm{in}}\right]\right\rangle_{[Y^{\mathrm{out}}, Y_q]} = \delta\left[Y^{\mathrm{out}}, Y_q\right] .$$

Now in the third equation we use that the contraction of basis encodings is the basis encoding of the composition function, and that $q \oplus q$ is the vanishing function, which basis encoding is a tensor product with $\epsilon_0\left[Y^{\mathrm{out}}_q\right]$. We use we Lem. 1 again in the fourth equation to arrive at the claim. □

When having a syntactical decomposition of a propositional formula, we can iteratively apply the computation circuit decomposition theorem and prepare each connective by a circuit. We can decompose any propositional formula into logical connectives and prepare to each a computation circuit (exploiting further sparsity mechanisms). This works, when the target qubit of one connective is used as a value qubit of another.

This entanglement of the working qubit with the distributed qubits can be resolved by an un-computation circuit, which is the adjoint of the computation circuit for the working qubits (see Figure 2). Note that while this is necessary in some applications, it is not necessary for the quantum rejection sampling presented here.

Example 4. *Let us reconsider the d-ary controlled* NOT *gate from Example 3. The d-ary logical conjunction is decomposed as a sequence of 2-ary logical conjunctions. The computation circuit of each 2-ary conjunction is a* Toffoli *gate applied to an input qubit and a worker qubit. Using Decomposition sparsity we therefore find the standard decomposition of d-ary controlled* NOT *gates into* Toffoli *gates.*

3.3 Polynomial sparsity

Lemma 3. *The composition of two computation circuits to functions q and g (see Figure 3), where the computation circuits share the target qubit, is the computation circuit of their sum mod2. In particular, computation circuits commute when they have the same target qubit.*

Proof. This follows from the representation of computation circuits by basis encodings of the corresponding function and the mod2 sum $\oplus$. □

Concatenating two computation circuits, which have the same head qubit, is the computation circuit of their mod2 sum.

A basis+ elementary function can be encoded by a single controlled NOT operation with auxiliary Pauli-X to prepare the control state at the X qubits.

This motivates the mod2-basis+ CP decomposition of tensors, which is exactly the decomposition of boolean polynomials into monomials. Each monomial is called a term (products of x or (1-x) factors), and minterms in case that all variables appear.

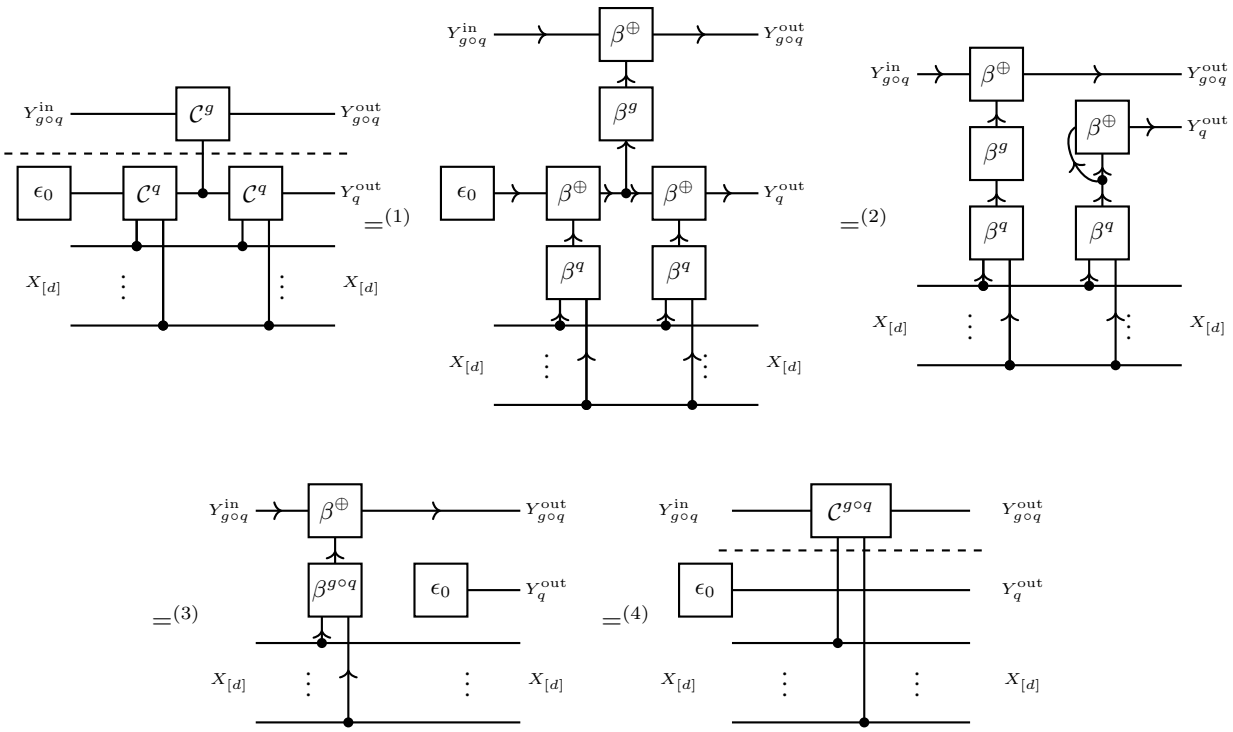

Figure 2: Proof of Lem. 2 based on basis calculus.

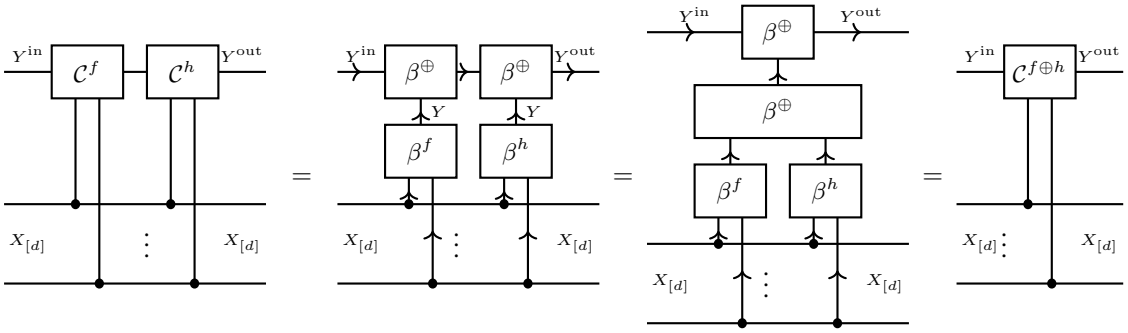

Figure 3: Equivalence of the composition of computation circuits with the computation circuit of their mod2 sum (Lem. 3).

Definition 4. *Given a boolean tensor τ, a mod2-basis+ CP decomposition is a collection $\mathcal{M}$ of tuples (A, x_A) with such that for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$*

$$\tau\left[X_{[d]} = x_{[d]}\right] = \bigoplus_{(A,x_A)\in\mathcal{M}} \langle \epsilon_{x_A}\left[X_A\right]\rangle_{\left[X_{[d]}=x_{[d]}\right]} .$$

This decomposition is closely related to the Algebraic Normal Form of a Boolean polynomial. These are called mixed polarity Reed-Muller expanions.

Using that basis CP decompositions are a special case of basis+ CP decompositions, we get the following rank bound.

Lemma 4. *The mod2-basis+ CP rank is bounded by the basis CP rank.*

Proof. Use $A = [d]$, and x_A to each supported state. Then the mod2-sum is a usual sum and the basis CP decomposition is also a mod2-basis+ CP decomposition. □

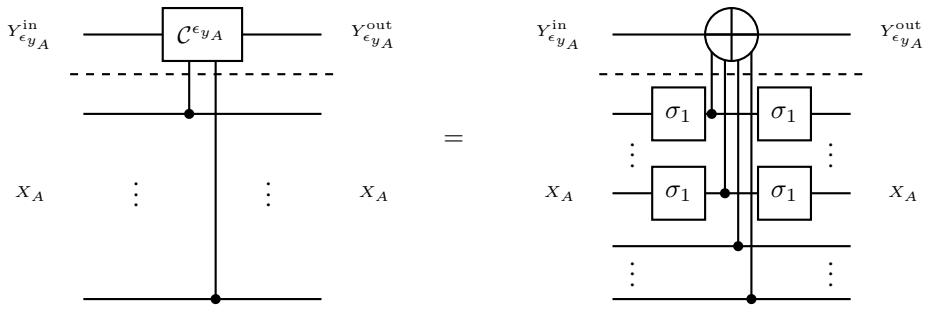

Figure 4: Representation of an slice as a controlled CNOT gate. When the control state to a variable is 0, we introduce auxiliary Pauli-X gates.

This shows in particular, that any propositional formula can be represented by a mod2-basis+ CP decomposition.

Lemma 5. *The computation circuit to a boolean tensor τ with a mod2-basis+ CP decomposition $\mathcal{M}$ obeys*

$$\begin{aligned}&\mathcal{C}^{\tau}\left[Y^{\text{in}}_{\tau}, Y^{\text{out}}_{\tau}, X_{[d]}\right]\\ &\quad= \left\langle \left\{\delta\left[Y^{\text{in}}_{\tau}, Y_0\right], \delta\left[Y^{\text{out}}_{\tau}, Y_{|\mathcal{M}|-1}\right]\right\} \cup \left\{\mathcal{C}^{\epsilon_{x_A}}\left[Y_i, Y_{i+1}, X_A\right] : (A, x_A) \in \mathcal{M}\right\}\right\rangle_{\left[Y^{\text{in}}_{\tau}, Y^{\text{out}}_{\tau}, X_{[d]}\right]}\end{aligned}$$

where $i \in [|\mathcal{M}|]$ enumerates the tuples in $\mathcal{M}$.

Proof. Can be seen by the basis encoding representation, and the accociativtiy of the sum mod2 ($\oplus$). □

Each computation circuit to each boolean monomial can be prepared by a multiple-controlled σ_1 gate and further pairs of σ_1 gates preparing the control state, see Figure 4. When we sum monomials wrt modulus 2 calculus, then the preparation is a sequence of such circuits. In such way, we can prepare the computation circuit to any propositional formula. This encoding strategy exploits a modified (by mod2 calculus) polynomial sparsity.

4 Activation circuits

We now study schemes to represent the activation network, which enable us to derive sampling schemes from CompActNets. To this end, we augment the activation network with ancilla variables, understanding it as a conditioned Bayesian network. We then apply controlled rotations to find q-samples.

4.1 Uniformly controlled rotations

Activation circuits are uniformly controlled unitaries (see Möttönen et al. (2005)), where the rotation axis is chosen as the y-axes of the Bloch sphere, and the angles are computed by the function $h(\cdot)$ acting on a function value to be encoded.

We define the angle preparing function on $p \in [0, 1]$ by

$$h(p) = 2 \cdot \cos^{-1}\left(\sqrt{1-p}\right) .$$

For any $p \in [0, 1]$ we then have

$$\left\langle \epsilon_0\left[A^{\text{in}}\right], R_Y\left(h(p)\right)\left[A^{\text{in}}, A^{\text{out}}\right]\right\rangle_{\left[A^{\text{out}}\right]} = \begin{pmatrix}\sqrt{1-p}\\ \sqrt{p}\end{pmatrix}\left[A^{\text{out}}\right] .$$

When we have a probability tensor, its activation circuit be prepared, since all values are in $[0, 1]$. Note that for rejection sampling, only the quotients of the values are important, we can therefore scale the value by a scalar such that the mode is 1.

Tensors $\tau\left[X_{[d]}\right]$ with non-negative coordinates can be encoded after dividing them by their maximum, that is the activation circuit of the function

$$f\left[x_{[d]}\right] = \frac{\tau\left[X_{[d]} = x_{[d]}\right]}{\max_{\tilde{y}_{[p]}} \tau\left[X_{[d]} = \tilde{y}_{[p]}\right]}$$

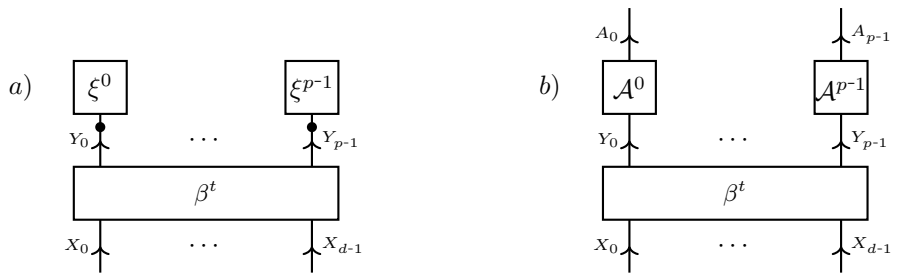

Figure 5: Ancilla augmentation of an elementary Computation-Activation Network a) by replacing each leg vector of the elementary activation tensor by an directed ancilla augmentation.

When the maximum of the tensor is not known, it can be replaced by an upper bound (reducing the acceptance rate of the rejection sampling).

4.2 Ancilla augmentation

The computation network consists already of directed cores and therefore is already a Bayesian network. The activation network however needs ancilla augmentation. Let us first define the ancilla augmentation of a distribution, and then find ancilla augmentations to CompActNets, which are Bayesian networks.

Definition 5 (Ancilla Augmentation of a Distribution)**.** *Let $\mathbb{P}\left[X_{[d]}\right]$ be a probability distribution of variables $X_{[d]}$. Another distribution $\tilde{\mathbb{P}}[X_{[d]}, A_{[p]}]$ of variables $X_{[d]}$ and ancilla variables $A_{[p]}$ is called an ancilla augmentation of $\mathbb{P}\left[X_{[d]}\right]$, if there is $\lambda > 0$ with*

$$\lambda \cdot \mathbb{P}\left[X_{[d]}\right] = \tilde{\mathbb{P}}[X_{[d]}, A_{[p]} = 1_{[p]}]\,.$$

We refer to λ as the corresponding acceptance probability.

Sampling from the distribution can be done by rejection sampling, also called post selection, on the ancilla augmented distribution in the following way. When drawing a sample $(x_{[d]}, a)$ from $\tilde{\mathbb{P}}[X_{[d]}, A]$ and rejecting it whenever $a = 0$, the accepted samples are distributed as $\mathbb{P}\left[X_{[d]}\right]$. We then have λ as the probability of accepting a sampling in this rejection sampling method. The acceptance probability is critical in determining the efficiency of this approach, since we need to draw $\sim \frac{1}{\lambda}$ samples from $\tilde{\mathbb{P}}[X_{[d]}, A]$ to get an accepted sample.

Definition 6. *The activation circuit of a non-negative tensor $\xi^e\left[Y_e\right]$ is the controlled unitary*

$$\mathcal{V}^{\xi^e}\left[A_e^{\text{in}}, A_e^{\text{out}}, Y_e\right] = \sum_{y_e} \epsilon_{y_e}\left[Y_e\right] \otimes R_Y\left(h\left(\frac{\xi^e\left[Y_e = y_e\right]}{\max_{\tilde{y}_e} \xi^e\left[Y_e = \tilde{y}_e\right]}\right)\right)\left[A^{\text{in}}, A^{\text{out}}\right]\,.$$

The activation circuit of a tensor network $\{\xi^e\left[Y_e\right] : e \in \mathcal{E}\}$ is the contraction of the activation tensors to each tensor in the network, that is

$$\mathcal{V}^{\{\xi^e[Y_e] : e \in \mathcal{E}\}}\left[A_{\mathcal{E}}^{\text{in}}, A_{\mathcal{E}}^{\text{out}}, Y_{\mathcal{V}}\right] = \left\langle \{\mathcal{V}^{\xi^e}\left[A_e^{\text{in}}, A_e^{\text{out}}, Y_e\right] : e \in \mathcal{E}\}\right\rangle_{\left[A_{\mathcal{E}}^{\text{in}}, A_{\mathcal{E}}^{\text{out}}, Y_{\mathcal{V}}\right]}\,.$$

Elementary Computation-Activation Networks can be augmented by ancilla augmentation of each leg vector in the activation tensor network (see Figure 5). We understand ancilla variables as variables in a Bayesian Network having single parents, corresponding with the ancilla augmentation of an elementary tensor. For an example, see Figure 7.

To a generic activation hypergraph $\mathcal{G}$, we would do ancilla augmentation for the activation tensor network, where each hidden variable is treated as a distributed qubit. To treat it as a distributed qubit, each hidden activation qubit is prepared in uniform state (i.e. a Hadamard gate acting on a ground state) and will be omitted in the measurement scheme (either not measured, or its measurement ignored). This can be understood from a tensor network perspective, where marginalization means contracting, which is exactly how hidden activation variables are used. For an example in the TT format, see Figure 6.

4.3 Q-samples of CompActNets

Theorem 1. *Given a CompActNet with statistic t and activation tensor network $\xi\left[Y_{[p]}\right]$, the circuit composed of*

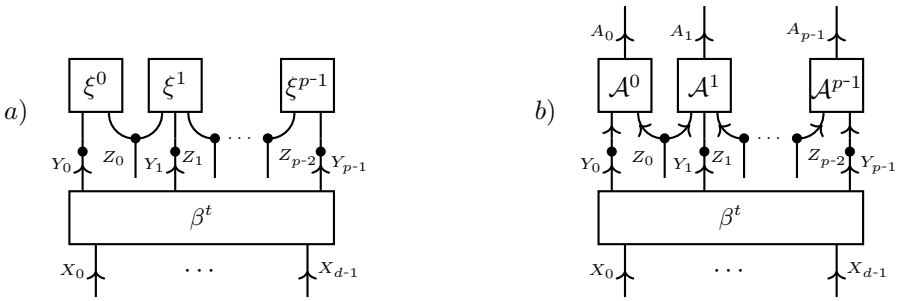

Figure 6: Ancilla augmentation of an TT Computation-Activation Network a) by a Bayesian Network b). The hidden variables in the TT activation tensor are treated as distributed variables. Marginalization over the hidden variables and conditioning on the activation variables in state 1, we reproduce the Computation-Activation Network.

- *Hadamard transforms on the distributed qubits* $X_{[d]}$
- *Computation circuit* $\mathcal{C}^t$
- *Activation circuit to* $\mathcal{V}^{\xi\left[Y_{[p]}\right]}$

prepares the q-sample of a distribution $\tilde{\mathbb{P}}[Y_{[p]}, X_{[d]}, A_{[p]}]$*, which is conditioned on* $A_{[p]} = 1_{[p]}$ *the CompActNet, that is*

$$\tilde{\mathbb{P}}[Y_{[p]}, X_{[d]}, A_{[p]} = 1_{[p]}] = \left\langle \beta^t \left[Y_{[p]}, X_{[d]}\right], \xi\left[Y_{[p]}\right]\right\rangle_{\left[Y_{[p]}, X_{[d]} \mid \varnothing\right]} .$$

Example 5 (Toy Accounting Example)**.** *For a more detailed example, let us consider a system of three variables* $A1$ *Account 1 is booked,* $A2$ *Account 2 is booked,* F *a feature on an invoice. Assume the following two rules have to be respected:*

- *Exactly one account must be booked.*
- *If feature* F *is present on the invoice, the account* A1 *is typically booked.*

We formalize this with the statistic

$$t = (X_{A1} \oplus X_{A2}, X_F \Rightarrow X_{A1}) .$$

Any elementary Computation-Activation Network with the statistic t *can be realized by the circuit shown in Figure 7. For the preparation of the first statistic qubit* Y_0 *to* $t_0 = X_{A1} \oplus X_{A2}$ *by a computation circuit we exploit that*

$$\begin{aligned} X_{A1} \oplus X_{A2} &= \epsilon_1\left[X_{A1}\right] \otimes \epsilon_0\left[X_{A2}\right] + \epsilon_0\left[X_{A1}\right] \otimes \epsilon_1\left[X_{A2}\right] \\ &= \left(\epsilon_1\left[X_{A1}\right] \otimes \epsilon_0\left[X_{A2}\right]\right) \oplus \left(\epsilon_0\left[X_{A1}\right] \otimes \epsilon_1\left[X_{A2}\right]\right) \end{aligned}$$

where by $\oplus$ *we denote coordinatewise summation mod 2. The statistic qubit* Y_0 *is therefore prepared by two controlled NOT gates (see Figure 7b).*

For the preparation of the second statistic qubit Y_1 *to* $t_1 = X_F \Rightarrow X_{A1}$ *we exploit that the implication is* True *except for the case where the premise* X_F *is* True *and the head* X_{A1} *is* False*. In out mod 2 calculus, this amounts to*

$$X_F \Rightarrow X_{A1} = \mathbb{I}\left[X_F, X_{A1}\right] \oplus \epsilon_1\left[X_F\right] \otimes \epsilon_0\left[X_{A1}\right] .$$

It follows that the statistic qubit Y_1 *is prepared by a NOT gate (that is a Pauli-X gate* σ_1*) and a second controlled NOT gate (see Figure 7b).*

For both cases, the preparation of the statistic qubit can be done by two controlled NOT gates exploiting the polynomial sparsity of the connectives.

Any non-elementary Computation-Activation Network with the statistic t *can be prepared by the circuit, when choosing a single ancilla qubit, which is uniformly controlled by the qubits* $Y_{[2]}$.

By the ancilla augmentation, a CompActNet is turned into a Bayesian network, which has additional sparse structure in its computation network. We can therefore understand the computation circuits and activation circuits as q-sample preparating circuits as in Low et al. (2014). The main difference is that we apply sparsity principles to further decompose the computation circuits.

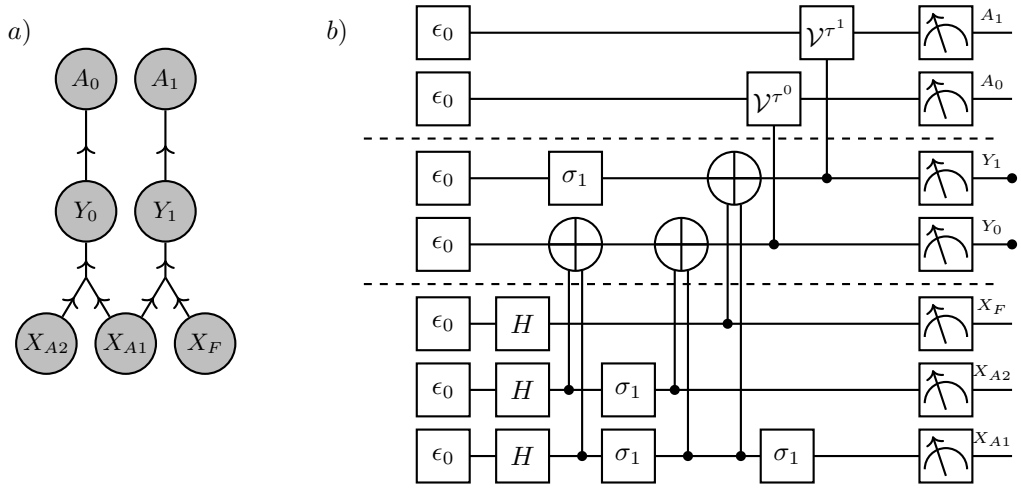

Figure 7: Circuit to sample from the Computation-Activation Network in the toy accounting Example 5.

5 Quantum Rejection Sampling from Computation-Activation Networks

In general quantum rejection sampling consist in two steps:

- Represent the distribution to be sampled as a conditionalization of a preparable distribution
- Amplify the amplitude of the ancilla qubits in state 1 by repeated Grover operators

We investigate, how the above circuit encoding schemes can be applied in the preparation of q-samples to Computation-Activation Networks. In particular, we build compositions of computation circuits and activation circuits, for which conditional distributions coincide with Computation-Activation Networks.

Both are defined using controlled single qubit gates (see Sections 4.2-3 in Nielsen (2010)) with ancilla qubits being the target qubits.

5.1 Amplitude Amplification

The above scheme prepares the ancilla augmented Computation-Activation Network

$$\tilde{\mathbb{P}}[A_{[p]}, Y_{[p]}, X_{[d]}] = \lambda \cdot \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \mathbb{P}\left[X_{[d]}\right]\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} + (1-\lambda)\mathbb{P}^{\perp}\left[X_{[d]}, Y_{[p]}, A_{[p]}\right]$$

and the prepared state is the combination of q-samples

$$\psi^0\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \sqrt{\lambda} \cdot \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \sqrt{\mathbb{P}\left[X_{[d]}\right]}\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} + \sqrt{(1-\lambda)}\sqrt{\mathbb{P}^{\perp}\left[X_{[d]}, Y_{[p]}, A_{[p]}\right]}\,.$$

Sampling from the augmented Computation-Activation Network can be also done classically by forward sampling: Just draw the variables from uniform, calculate the value qubit by a logical circuit inference and accept with probability by the computed value.

To achieve a quantum advantage we need amplitude amplification on the ancilla qubits.

In the subspace V spanned by the axes

- $\psi^{\parallel} = \epsilon_{1_{[p]}}\left[A_{[p]}\right] \otimes \left\langle \beta^t\left[Y_{[p]}, X_{[d]}\right], \sqrt{\mathbb{P}\left[X_{[d]}\right]}\right\rangle_{\left[Y_{[p]}, X_{[d]}\right]}$
- $\psi^{\perp} = \sqrt{\mathbb{P}^{\perp}}$

which can be thought of the polar angle coordinate of the state

$$\psi^0\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \sin\left(\frac{\alpha}{2}\right)\psi^{\parallel}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] + \cos\left(\frac{\alpha}{2}\right)\psi^{\perp}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right]$$

where we define an angle α by

$$\sin\left(\frac{\alpha}{2}\right) := \sqrt{\lambda}$$

The amplitude amplification scheme now iterates between

- Reflection $\mathcal{S}^{\parallel}$ along $\psi^{\parallel}$, by

$$\mathcal{S}^{\parallel}\left[A^{\text{in}}_{[p]}, A^{\text{out}}_{[p]}\right] = \delta\left[A^{\text{in}}_{[p]}, A^{\text{out}}_{[p]}\right] - 2\epsilon_{1_{[p]}}\left[A^{\text{in}}_{[p]}\right] \otimes \epsilon_{1_{[p]}}\left[A^{\text{out}}_{[p]}\right]$$

- Reflection $\mathcal{S}^{\psi}$ along ψ, by

$$\begin{aligned}\mathcal{S}^{\psi}\left[A^{\text{in}}_{[p]}, Y^{\text{in}}_{[p]}, A^{\text{in}}_{[p]}, A^{\text{out}}_{[p]}, Y^{\text{out}}_{[p]}, A^{\text{out}}_{[p]}\right] =& \delta\left[A^{\text{in}}_{[p]}, Y^{\text{in}}_{[p]}, A^{\text{in}}_{[p]}, A^{\text{out}}_{[p]}, Y^{\text{out}}_{[p]}, A^{\text{out}}_{[p]}\right] \\ &- 2\psi^{0}\left[A^{\text{in}}_{[p]}, Y^{\text{in}}_{[p]}, X^{\text{in}}_{[d]}\right] \otimes \psi^{0}\left[A^{\text{out}}_{[p]}, Y^{\text{out}}_{[p]}, X^{\text{out}}_{[d]}\right]\end{aligned}$$

  This reflection can be done by concatenating the operator preparing the ancilla-augmentation of the Computation-Activation Network with a initial state reflection.

Now the application of j grover operators $\mathcal{S}\mathcal{S}^{\parallel}$ on ψ^0 prepares a state (see Figure 8)

$$\psi^{j}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] = \sin\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\parallel}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right] + \sin\left(\left(\frac{1}{2}+j\right)\alpha\right)\psi^{\perp}\left[A_{[p]}, Y_{[p]}, X_{[d]}\right]$$

The state is closest to $\psi^{\parallel}$ when

$$\left(\frac{1}{2}+j\right)\alpha \approx \frac{\pi}{2}\,.$$

Estimating for small λ (tight for small λ)

$$\sqrt{\lambda} = \sin\left(\frac{\alpha}{2}\right) \leq \frac{\alpha}{2}$$

we get a bound

$$j^{\text{opt}} \leq \frac{\pi}{4\sqrt{\lambda}}\,.$$

This shows that we have a square root improvement on classical rejection sampling, which would require $\frac{1}{\lambda}$ repetitions to produce a sample. Note however that we compare quantum circuit length with the number of classical repetitions in sampling.

5.2 Sampling from Computation-Activation Networks

For Computation-Activation Networks we find a q-sample by concatenating computation circuits and activation circuits (see Figure 9).

6 Conclusion and Outlook

We have characterized an intuitive class of Quantum Circuits, namely graph-controlled circuits. We have shown that their prepared distributions are exactly the Bayesian Networks on the corresponding hypergraph. Along this class the demonstrated rejection sampling schemes by Computation Activation Circuits achieve the best acceptance probability. Using amplitude amplification, we get a square root improvement of the quantum rejection sampling scheme compared with classical rejection sampling.

To improve on this quantum speedup, different circuit designs are necessary. We could allow for hypergraphs with cycles, where e.g. the distributed qubits are rotated conditioned on the statistic qubits. Such schemes shall be investigated in future research. However, an improvement beyond scaling constants is not expected, since the amplitude amplification procedure already achieves the Heisenberg limit Brassard et al. (2002); **?**.

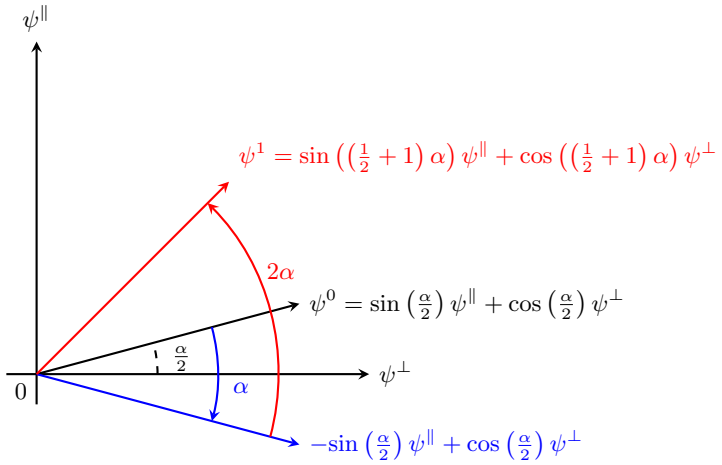

Figure 8: Sketch of the amplitude amplification scheme.

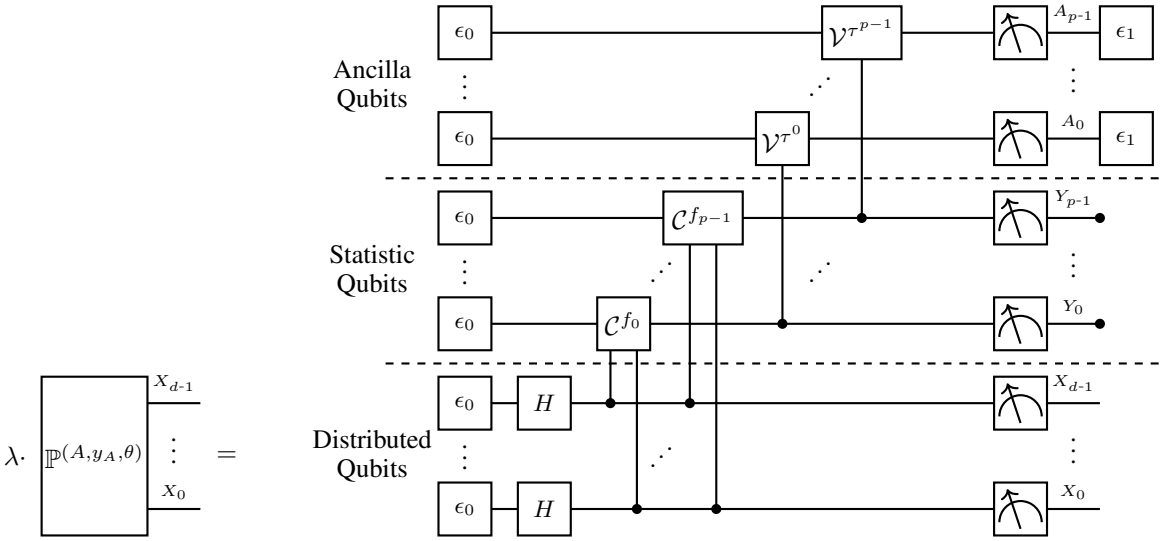

Figure 9: Quantum Circuit to reproduce a q-sample of an augmented Computation-Activation Network. We measure the distributed qubits $X_{[d]}$ and the ancilla qubits $A_{[p]}$ and reject all samples, where an ancilla qubit is measured as 0.

Our quantum rejection sampling scheme can be used as a subroutine in variational inference schemes, for example when estimating the parameters of the Computation-Activation Network. When sampling from probability distributions, we can use these samples to estimate probabilistic queries, i.e. perform forward inference. Building on such particle-based forward inference schemes in inner loops, we can perform various inference schemes for Computation-Activation Networks, such as backward inference and message passing schemes.

One might further investigate the relation between amplitude amplification and change of the canonical parameters in Hybrid Logic Network. This will enable a more integrated alternative to the Parameter Estimation routines suggested here. Furthermore it will enable the preparation of Computation-Activation Networks without the need of ancilla augmentation. While we expect limits due to the discrete degrees of freedom of amplitude amplification, we could refine prepared circuits based on ancilla augmentation. To this end, we would understand such preparation as a coarse representation of Computation-Activation Networks, which serve as initial states for ancilla augmentation. The advantage then lies in the improvement of the λ, when the ∞ Renyi divergence with respect to the coarse distribution is less than the ∞ Renyi divergence with respect to the uniform distribution.

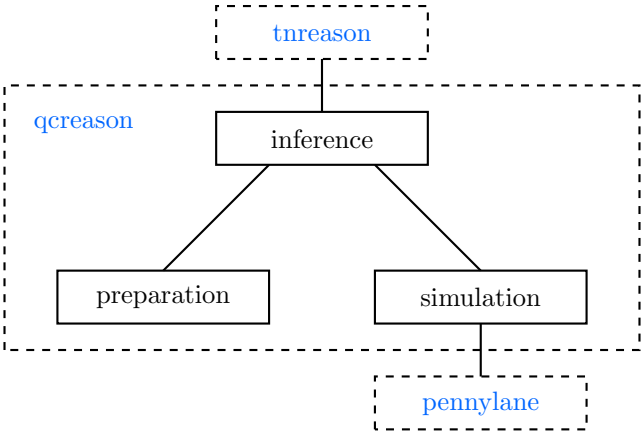

Figure 10: Architecture of the qcreason package. The inference sub-package receives inference tasks from tnreason and calls preparation and simulation for corresponding circuit preparation and simulation.

A Implementation

The introduced quantum circuit preparation schemes have been implemented in the python package qcreason, which architecture is sketched in Figure 10.

A.1 Quantum Circuits

Quantum Circuits are stored as lists of dictionaries, where each dictionary represents a quantum gate application. We specify the unitaries by strings accessible via the key *"unitary"*, as listed in the following table:

Unitary	String Representation	Parameters	Tensor Notation
Hadamard	*"H"*		$H\left[X^{\text{in}}, X^{\text{out}}\right]$
Pauli-X	*"X"*		$\sigma_1\left[A^{\text{in}}, A^{\text{out}}\right]$
Rotation around Y-axis	*"RY"*	Angle α	$R_Y(\alpha)\left[A^{\text{in}}, A^{\text{out}}\right]$

The control in controlled operations is specified by a dictionary accessible via the key *"control"*. It contains as keys the control qubits and their control states as values (i.e. the state in which the unitary is applied).

Further parameters such as angles are added in another dictionary, accessible via the key *"parameters"*.

For example a hadamard gate on qubit *"blue"* and a multiple controlled rotation around the Y-axis on qubit *"ancilla_c1"* controlled by qubits *"red"* and *"blue"* in state $(0, 1)$ with an angle is represented as:

```
[{'unitary': 'H',
    'target': ['blue']},
{'unitary': 'MCRY',
    'target': ['ancilla_c1'],
    'control': {'red': 1, 'blue': 0},
    'parameters': {'angle': np.float64(0.6121442808462428)}}]
```

Note that we omit the notation of incoming and outgoing variables.

A.2 Generic Contraction

So far, the generic contraction routine consists in activation circuits preparing for each tensor an ancilla variable, and post-selecting the measurements where the ancilla variable is 1. The resulting tensor of the contraction is a PandasCore storing the post-selected measurement results in a dataframe.

References

Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and Weaknesses of Quantum Computing. *SIAM Journal on Computing*, 26(5):1510–1523, October 1997. ISSN 0097-5397, 1095-7111. doi: 10.1137/S0097539796300933. URL http://arxiv.org/abs/quant-ph/9701001. arXiv:quant-ph/9701001.

Gilles Brassard and Peter Hoyer. An Exact Quantum Polynomial-Time Algorithm for Simon's Problem. In *Proceedings of the Fifth Israel Symposium on the Theory of Computing Systems (ISTCS '97)*, ISTCS '97, page 12, USA, June 1997. IEEE Computer Society. ISBN 978-0-8186-8037-3.

Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. Quantum Amplitude Amplification and Estimation, 2002. URL http://arxiv.org/abs/quant-ph/0005055. arXiv:quant-ph/0005055.

Marcus Cramer, Martin B. Plenio, Steven T. Flammia, Rolando Somma, David Gross, Stephen D. Bartlett, Olivier Landon-Cardinal, David Poulin, and Yi-Kai Liu. Efficient quantum state tomography. *Nature Communications*, 1 (1):149, December 2010. ISSN 2041-1723. doi: 10.1038/ncomms1147. URL https://www.nature.com/articles/ncomms1147.

Alex Goessmann. The Tensor-Network Approach to Efficient and Explainable AI, 2025. URL https://github.com/EnexaProject/enexa-tensor-reasoning-documentation/.

Alex Goessmann, Janina Schütte, Maximilian Fröhlich, and Martin Eigel. A tensor network formalism for neuro-symbolic AI, January 2026. URL http://arxiv.org/abs/2601.15442. arXiv:2601.15442 [cs].

David Gross, Yi-Kai Liu, Steven T. Flammia, Stephen Becker, and Jens Eisert. Quantum State Tomography via Compressed Sensing. *Physical Review Letters*, 105(15):150401, October 2010. doi: 10.1103/PhysRevLett.105.150401. URL https://link.aps.org/doi/10.1103/PhysRevLett.105.150401.

Lov Grover and Terry Rudolph. Creating superpositions that correspond to efficiently integrable probability distributions, August 2002. URL http://arxiv.org/abs/quant-ph/0208112. arXiv:quant-ph/0208112.

Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of Computing*, STOC '96, pages 212–219, New York, NY, USA, July 1996. Association for Computing Machinery. ISBN 978-0-89791-785-8. doi: 10.1145/237814.237866. URL https://dl.acm.org/doi/10.1145/237814.237866.

Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. The MIT Press, Cambridge, Mass., 1. edition edition, July 2009. ISBN 978-0-262-01319-2.

Guang Hao Low, Theodore J. Yoder, and Isaac L. Chuang. Quantum inference on Bayesian networks. *Physical Review A*, 89(6):062315, June 2014. doi: 10.1103/PhysRevA.89.062315. URL https://link.aps.org/doi/10.1103/PhysRevA.89.062315.

Mikko Möttönen, Juha J. Vartiainen, Ville Bergholm, and Martti M. Salomaa. Transformation of quantum states using uniformly controlled rotations. *Quantum Info. Comput.*, 5(6):467–473, September 2005. ISSN 1533-7146.

Isaac L. Chuang Michael A. Nielsen. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, Cambridge ; New York, December 2010. ISBN 978-1-107-00217-3.

Maris Ozols, Martin Roetteler, and Jérémie Roland. Quantum rejection sampling. *ACM Trans. Comput. Theory*, 5(3): 11:1–11:33, August 2013. ISSN 1942-3454. doi: 10.1145/2493252.2493256. URL https://doi.org/10.1145/2493252.2493256.

Ingo Roth, Jadwiga Wilkens, Dominik Hangleiter, and Jens Eisert. Semi-device-dependent blind quantum tomography. *Quantum*, 7:1053, July 2023. doi: 10.22331/q-2023-07-11-1053. URL https://quantum-journal.org/papers/q-2023-07-11-1053/.

Maria Schuld and Francesco Petruccione. *Machine Learning with Quantum Computers*. Springer, Cham, October 2021. ISBN 978-3-030-83097-7.

Peter Wittek and Christian Gogolin. Quantum Enhanced Inference in Markov Logic Networks. *Scientific Reports*, 7(1): 45672, April 2017. ISSN 2045-2322. doi: 10.1038/srep45672. URL http://arxiv.org/abs/1611.08104.